Bài 2.5 – Đánh giá phân loại: Confusion Matrix, Accuracy, Precision, Recall, F1, ROC-AUC

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *Câu chuyện*

🎯 Sau bài này bạn sẽ

- Đọc thành thạo **Confusion Matrix**, biết 4 loại đúng/sai.
- Hiểu vì sao **Accuracy có thể lừa bạn**.
- Tính tay và chọn đúng giữa **Precision, Recall, F1** theo bài toán kinh doanh.
- Biết **dịch ngưỡng** quyết định để cân bằng các loại lỗi theo **chi phí thật**.
- Đọc **đường ROC** và **AUC**: đánh giá mô hình không phụ thuộc ngưỡng.

Chia buổi:

- **Buổi 1:** Mục 1 → 5 (confusion matrix, accuracy, precision, recall, F1).
- **Buổi 2:** Mục 6 → 10 (ngưỡng, chi phí, ROC-AUC, bài tập).

---

1. Câu chuyện: Hệ thống báo trộm "chính xác 99,9%"

Một công ty bán hệ thống camera báo trộm quảng cáo: *"Độ chính xác 99,9%!"*

Bạn lắp thử. Một năm có 10.000 lần có người đi qua cửa nhà bạn, trong đó có **10 lần là trộm thật**. Bạn phát hiện hệ thống **không bao giờ báo động**, kể cả 10 lần có trộm.

Độ chính xác? (10.000 − 10) / 10.000 = **99,9%**. Công ty không nói dối!

Một hệ thống **không làm gì cả** vẫn đạt 99,9%. Vì trộm quá hiếm (giống bệnh hiếm ở bài 1.10), "luôn nói không có trộm" gần như luôn đúng. Đây gọi là **nghịch lý độ chính xác (accuracy paradox)**.

Với bài toán chấm điểm tín dụng (30% vỡ nợ), ngân hàng cần những con số **không bị lừa** như vậy. Bài này trang bị cho bạn chúng.

---

2. Chuẩn bị: mô hình từ bài 2.4

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
credit["vo_no"] = (credit["class"] == "bad").astype(int)
X = pd.get_dummies(credit[["duration", "credit_amount", "age",
"checking_status"]],
```

```
                    columns=["checking_status"], drop_first=True, dtype=int)
y = credit["vo_no"]
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
sc = StandardScaler().fit(Xtr)
Xtr_s, Xte_s = sc.transform(Xtr), sc.transform(Xte)
mh = LogisticRegression().fit(Xtr_s, ytr)

p = mh.predict_proba(Xte_s)[:, 1]          # xác suất vỡ nợ của từng khách trong
test
du_doan = (p >= 0.5).astype(int)           # quyết định với ngưỡng 0.5
print(yte.value_counts().to_dict())        # tập test có bao nhiêu khách mỗi loại
```

Kết quả mẫu:

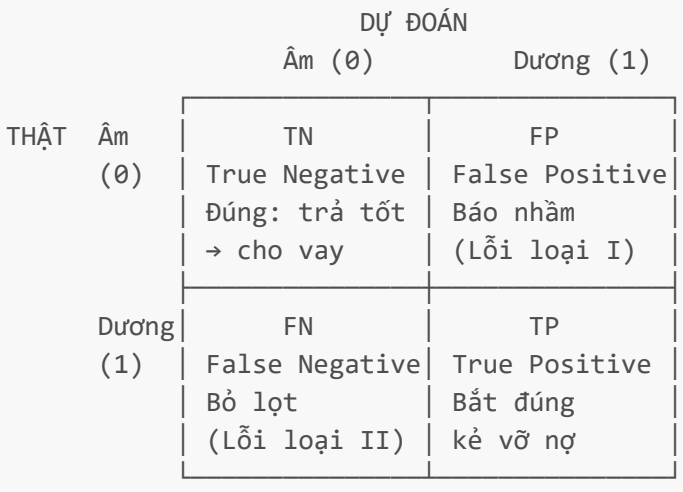
```
{0: 140, 1: 60}
```

---

3. Confusion Matrix (Ma trận nhầm lẫn)

3.1. Khái niệm

- **Định nghĩa:** Bảng đếm có bao nhiêu mẫu rơi vào mỗi cặp (nhãn thật, nhãn dự đoán).
- **Ví dụ đời thực:** Bảng tổng kết sau một mùa kiểm tra an ninh sân bay: bắt đúng bao nhiêu người mang đồ cấm, chặn nhầm bao nhiêu người vô tội...



Mẹo nhớ tên: chữ thứ hai (Positive/Negative) là **mô hình nói gì**. Chữ thứ nhất (True/False) là **mô hình nói đúng hay sai**.

- **FP** = mô hình nói **Dương**, và nói **Sai** → khách tốt bị từ chối oan.
- **FN** = mô hình nói **Âm**, và nói **Sai** → kẻ vỡ nợ lọt lưới.

3.2. Code

```
from sklearn.metrics import confusion_matrix

print(confusion_matrix(yte, du_doan))                 # hàng: thật; cột: dự đoán
tn, fp, fn, tp = confusion_matrix(yte, du_doan).ravel()   # ravel: dàn thành 4 số
print(tn, fp, fn, tp)
```

Kết quả mẫu:

```
[[129  11]
 [ 41  19]]
129 11 41 19
```

Đọc: Trong 60 khách thật sự vỡ nợ, mô hình chỉ bắt được **19**, để lọt **41**. Trong 140 khách tốt, từ chối nhầm 11.

---

4. Accuracy và nghịch lý

```
                TP + TN
Accuracy = ──────────────    = tỉ lệ đoán đúng trên tổng số
              tổng số
```

Tính: (19 + 129) / 200 = 148 / 200 = **0,74**.

So với mốc "luôn đoán trả tốt":

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score

moc = DummyClassifier(strategy="most_frequent").fit(Xtr_s, ytr)   # luôn đoán lớp phổ biến nhất
d_moc = moc.predict(Xte_s)
print(accuracy_score(yte, d_moc), recall_score(yte, d_moc),
      precision_score(yte, d_moc, zero_division=0))   # zero_division=0: tránh cảnh báo chia cho 0
```

Kết quả mẫu:

```
0.7 0.0 0.0
```

Mốc "không làm gì" đạt **70%**, chỉ kém mô hình 4 điểm phần trăm, trong khi **không bắt được ai** (recall = 0). Accuracy **không phù hợp** khi các lớp mất cân bằng. Ta cần thước đo nhìn riêng vào lớp dương.

---

5. Precision, Recall, F1

5.1. Precision (Độ chính xác của dự đoán dương)

- **Câu hỏi:** "Trong những người **mô hình báo** là vỡ nợ, bao nhiêu % **thật sự** vỡ nợ?"
- **Ví dụ đời thực:** Bộ lọc spam: trong những email bị đưa vào thư mục Spam, bao nhiêu % thật là spam? Precision thấp = **email quan trọng bị chôn oan** trong Spam.

$$\text{Precision} = \frac{TP}{TP + FP}$$

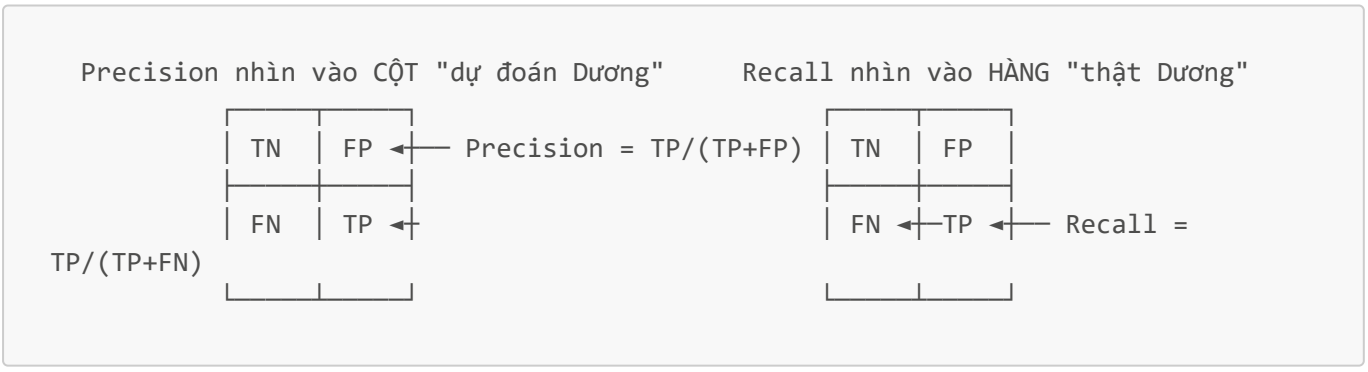
Tính: 19 / (19 + 11) = 19 / 30 ≈ **0,633**.

5.2. Recall (Độ phủ / Độ nhạy)

- **Câu hỏi:** "Trong những người **thật sự** vỡ nợ, mô hình **bắt được** bao nhiêu %?"
- **Ví dụ đời thực:** Xét nghiệm ung thư: trong những người có bệnh, xét nghiệm phát hiện được bao nhiêu %? Recall thấp = **bỏ sót người bệnh**.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Tính: 19 / (19 + 41) = 19 / 60 ≈ **0,317**.



5.3. F1-score

- **Định nghĩa:** Trung bình **điều hòa** của Precision và Recall. Cao khi **cả hai** cùng cao.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Tính: 2 × 0,633 × 0,317 / (0,633 + 0,317) = 0,401 / 0,950 ≈ **0,422**.

Vì sao không dùng trung bình cộng? Precision = 1, Recall = 0 (bắt 1 người đúng, bỏ sót tất cả còn lại) → trung bình cộng = 0,5, nghe ổn. F1 = 0: phản ánh đúng là mô hình **vô dụng**. Trung bình điều hòa bị kéo về phía số **nhỏ hơn**.

5.4. Code

```
from sklearn.metrics import f1_score, classification_report

print(accuracy_score(yte, du_doan), round(precision_score(yte, du_doan), 3),
      round(recall_score(yte, du_doan), 3), round(f1_score(yte, du_doan), 3))
print(classification_report(yte, du_doan, target_names=["Trả tốt", "Vỡ nợ"]))
```

Kết quả mẫu:

```
0.74 0.633 0.317 0.422
              precision    recall  f1-score   support

     Trả tốt       0.76      0.92      0.83       140
       Vỡ nợ       0.63      0.32      0.42        60

    accuracy                           0.74       200
   macro avg       0.70      0.62      0.63       200
weighted avg       0.72      0.74      0.71       200
```

Đọc `classification_report`:

- Mỗi lớp có precision, recall, F1 **riêng**. `support` = số mẫu thật của lớp đó.
- **macro avg**: trung bình cộng các lớp, coi lớp nào cũng quan trọng như nhau.
- **weighted avg**: trung bình có trọng số theo `support`, bị lớp đông áp đảo.
- Mô hình giỏi nhận ra người **trả tốt** (recall 0,92) nhưng rất kém nhận ra người **vỡ nợ** (recall 0,32). Với ngân hàng, đó là ngược với điều họ cần!

5.5. Precision hay Recall? Phụ thuộc chi phí lỗi

Bài toán	Lỗi nào đắt hơn?	Ưu tiên
Sàng lọc ung thư	Bỏ sót người bệnh (FN)	**Recall**
Phát hiện gian lận thẻ	Bỏ lọt gian lận (FN)	**Recall** (rồi người thật kiểm tra lại)
Lọc spam	Chôn nhầm email quan trọng (FP)	**Precision**
Gợi ý video "chắc chắn bạn thích"	Gợi ý sai làm người dùng chán (FP)	**Precision**
Chấm điểm tín dụng	Cho kẻ vỡ nợ vay (FN) **thường đắt hơn** từ chối nhầm (FP)	Cân bằng theo chi phí (mục 7)

6. Ngưỡng quyết định: không phải lúc nào cũng là 0,5

Mô hình cho ra **xác suất**. Ngưỡng 0,5 chỉ là **mặc định**, không phải luật. Hạ ngưỡng → báo "vỡ nợ" nhiều hơn → recall tăng, precision giảm.

```
for t in [0.2, 0.3, 0.4, 0.5, 0.6]:
    d = (p >= t).astype(int)                                    # quyết định theo
ngưỡng t
    tn, fp, fn, tp = confusion_matrix(yte, d).ravel()
    print(f"Ngưỡng {t}: TP={tp:>2} FP={fp:>3} FN={fn:>2} TN={tn:>3} | "
          f"Precision={precision_score(yte, d):.2f} Recall={recall_score(yte,
d):.2f} "
          f"F1={f1_score(yte, d):.2f} | Chi phí={5 * fn + fp}")
```

Kết quả mẫu:

```
Ngưỡng 0.2: TP=54 FP= 75 FN= 6 TN= 65 | Precision=0.42 Recall=0.90 F1=0.57 | Chi
phí=105
Ngưỡng 0.3: TP=49 FP= 54 FN=11 TN= 86 | Precision=0.48 Recall=0.82 F1=0.60 | Chi
phí=109
Ngưỡng 0.4: TP=43 FP= 23 FN=17 TN=117 | Precision=0.65 Recall=0.72 F1=0.68 | Chi
phí=108
Ngưỡng 0.5: TP=19 FP= 11 FN=41 TN=129 | Precision=0.63 Recall=0.32 F1=0.42 | Chi
phí=216
Ngưỡng 0.6: TP=13 FP=  2 FN=47 TN=138 | Precision=0.87 Recall=0.22 F1=0.35 | Chi
phí=237
```

```
ngưỡng thấp ◄──────────────────────────────► ngưỡng cao
"Nghi ngờ tất cả"                            "Chỉ báo khi rất chắc"
Recall cao, Precision thấp                   Precision cao, Recall thấp
(bắt được hầu hết, nhưng oan nhiều người)    (ít oan, nhưng lọt nhiều)
```

Phát hiện: Chỉ hạ ngưỡng từ 0,5 xuống 0,4, recall nhảy từ 0,32 lên **0,72**, F1 từ 0,42 lên **0,68**. Cùng một mô hình, **không huấn luyện lại**, chỉ đổi cách ra quyết định!

---

7. Chọn ngưỡng theo chi phí thật

Tài liệu gốc của bộ German Credit cho một **ma trận chi phí**: cho kẻ vỡ nợ vay (FN) gây thiệt hại **gấp 5 lần** từ chối nhầm khách tốt (FP).

```
Chi phí = 5 × FN + 1 × FP
```

Nhìn cột "Chi phí" ở bảng mục 6:

- Ngưỡng 0,5 (mặc định): chi phí **216**.
- Ngưỡng 0,2: chi phí **105**, **giảm hơn một nửa**.

Bài học quan trọng nhất của bài: Chọn ngưỡng (và chọn chỉ số đánh giá) là **quyết định kinh doanh**, không phải quyết định kỹ thuật. Hãy hỏi: "Mỗi loại lỗi tốn bao nhiêu tiền?"

> ⚠️ Ở đây ta chọn ngưỡng nhìn trên **tập test**: đó là gian lận nhẹ (bài 1.11). Làm đúng: chọn ngưỡng trên **validation** hoặc bằng cross-validation (bài 2.17), rồi mới báo cáo trên test.

---

8. ROC và AUC: đánh giá không phụ thuộc ngưỡng

8.1. ROC curve (Đường cong ROC)

- **Định nghĩa:** Vẽ **mọi ngưỡng** có thể lên một biểu đồ:
  - Trục y: **TPR – True Positive Rate** = Recall = TP / (TP + FN).
  - Trục x: **FPR – False Positive Rate** = FP / (FP + TN): tỉ lệ khách tốt bị báo nhầm.

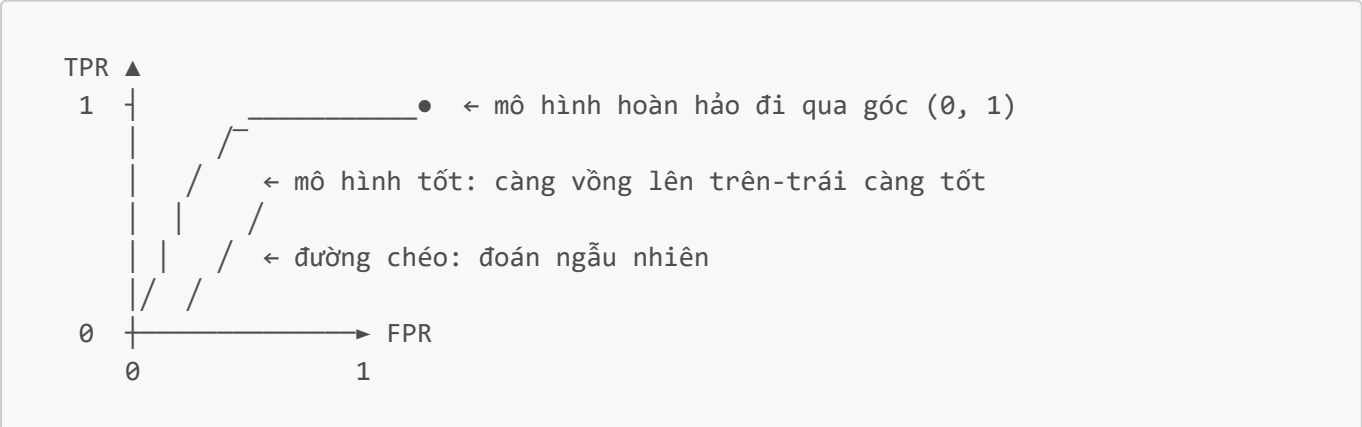
```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score

fpr, tpr, nguong = roc_curve(yte, p)          # tính TPR, FPR tại mọi ngưỡng
fig, ax = plt.subplots(figsize=(5, 5))
ax.plot(fpr, tpr, label=f"Logistic (AUC = {roc_auc_score(yte, p):.3f})")
ax.plot([0, 1], [0, 1], "--", label="Đoán ngẫu nhiên (AUC = 0.5)")
ax.set_xlabel("FPR (tỉ lệ khách tốt bị báo nhầm)")
ax.set_ylabel("TPR = Recall (tỉ lệ kẻ vỡ nợ bị bắt)")
ax.legend(); plt.show()
print(round(roc_auc_score(yte, p), 3))
```

Kết quả mẫu:

```
0.803
```

Bạn sẽ thấy: đường cong xanh vồng lên phía **góc trên bên trái**, nằm trên đường chéo nét đứt. Ví dụ ở FPR ≈ 0,17 (báo nhầm 17% khách tốt), TPR đã đạt khoảng 0,72 (bắt được 72% kẻ vỡ nợ).



8.2. AUC – Area Under the Curve (Diện tích dưới đường cong)

- **Định nghĩa:** Diện tích dưới đường ROC, từ 0 đến 1. Hoàn hảo = 1, đoán ngẫu nhiên = 0,5.
- **Ý nghĩa dễ hiểu nhất:** Chọn ngẫu nhiên **một khách vỡ nợ** và **một khách tốt**. AUC = xác suất mô hình cho **khách vỡ nợ điểm rủi ro cao hơn**.

Kiểm chứng ý nghĩa đó bằng cách so **mọi cặp**:

```
p_vo_no = p[yte.values == 1]          # điểm rủi ro của 60 khách vỡ nợ
p_tot = p[yte.values == 0]            # điểm rủi ro của 140 khách tốt
# So mọi cặp (60 × 140 = 8.400 cặp): khách vỡ nợ có điểm cao hơn không?
print(round((p_vo_no[:, None] > p_tot[None, :]).mean(), 3))   # None: thêm chiều
để broadcasting (bài 1.3)
```

Kết quả mẫu:

```
0.803
```

Khớp chính xác với AUC. Trong 80% số cặp, mô hình **xếp hạng đúng** ai rủi ro hơn.

AUC	Đánh giá (tham khảo)
0,5	Không hơn tung đồng xu
0,7 – 0,8	Chấp nhận được
0,8 – 0,9	Tốt
> 0,9	Rất tốt (hoặc nghi rò rỉ dữ liệu!)

Khi nào dùng AUC? Khi muốn so sánh các mô hình **trước khi** chọn ngưỡng, hoặc khi ngưỡng sẽ thay đổi theo chính sách. Ngân hàng rất hay dùng AUC (và chỉ số Gini = 2 × AUC − 1).

> Khi lớp dương **cực hiếm** (gian lận 0,1%), ROC-AUC có thể trông đẹp một cách dễ dãi. Khi đó nên xem thêm **đường Precision-Recall** (`precision_recall_curve`, `average_precision_score`).

---

9. Tổng hợp: chọn chỉ số nào?

Tình huống	Chỉ số
Lớp cân bằng, mọi lỗi như nhau	Accuracy
Lớp mất cân bằng, cần một con số chung	F1, AUC
Bỏ sót rất đắt (bệnh, gian lận)	Recall (kèm precision tối thiểu)
Báo nhầm rất đắt (spam, kiện tụng)	Precision (kèm recall tối thiểu)
So sánh mô hình không phụ thuộc ngưỡng	ROC-AUC

Tình huống	Chỉ số
Biết chi phí từng loại lỗi	**Tổng chi phí** (tốt nhất!)

---

10. Bài tập

1. **Tính tay.** 10 bệnh nhân. Nhãn thật: `[1,1,1,1,0,0,0,0,0,0]`. Dự đoán: `[1,1,1,0,1,1,0,0,0,0]`. Lập confusion matrix, tính Accuracy, Precision, Recall, F1. Kiểm tra bằng sklearn.
2. Một mô hình phát hiện gian lận có Precision = 0,9, Recall = 0,1. Mô tả bằng lời nó hoạt động thế nào. Có tốt không?
3. Nếu chi phí FN chỉ **gấp 2** chi phí FP, ngưỡng nào trong bảng mục 6 tốt nhất?
4. Vì sao AUC không thay đổi khi bạn đổi ngưỡng từ 0,5 sang 0,3?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. TP = 3, FN = 1, FP = 2, TN = 4. Accuracy = 7/10 = **0,7**. Precision = 3/5 = **0,6**. Recall = 3/4 = **0,75**. F1 = 2 × 0,6 × 0,75 / 1,35 ≈ **0,667**.

```
ty = [1,1,1,1,0,0,0,0,0,0]; dd = [1,1,1,0,1,1,0,0,0,0]
print(confusion_matrix(ty, dd))                    # [[4 2] [1 3]]
print(precision_score(ty, dd), recall_score(ty, dd), round(f1_score(ty, dd), 3))
# 0.6 0.75 0.667
```

2. Khi nó báo gian lận thì 90% là đúng (rất ít báo nhầm), nhưng nó **chỉ bắt được 10%** số vụ gian lận, để lọt 90%. Với ngân hàng, đây là mô hình **rất tệ**: quá rụt rè. Nên hạ ngưỡng để tăng recall.
3. Chi phí = 2 × FN + FP. Ngưỡng 0,2: 2×6 + 75 = 87. Ngưỡng 0,3: 22 + 54 = 76. Ngưỡng 0,4: 34 + 23 = **57**. Ngưỡng 0,5: 82 + 11 = 93. Ngưỡng 0,6: 94 + 2 = 96. → **Ngưỡng 0,4** tốt nhất. Chi phí khác → ngưỡng tối ưu khác.
4. AUC được tính từ **mọi ngưỡng cùng lúc** (cả đường cong). Nó chỉ phụ thuộc vào cách mô hình **xếp hạng** các khách, không phụ thuộc ngưỡng bạn chọn để ra quyết định.

---

11. Flashcard

Mặt trước	Mặt sau
FP và FN là gì?	FP: báo dương nhưng sai (báo nhầm). FN: báo âm nhưng sai (bỏ lọt).
Accuracy paradox?	Lớp mất cân bằng: đoán toàn lớp đông vẫn được accuracy cao.
Công thức Precision?	TP / (TP + FP): trong các báo dương, bao nhiêu đúng.
Công thức Recall?	TP / (TP + FN): trong các dương thật, bắt được bao nhiêu.
F1 là gì?	Trung bình điều hòa của Precision và Recall.
Hạ ngưỡng thì sao?	Recall tăng, Precision giảm.

Mặt trước	Mặt sau
Chọn ngưỡng dựa vào đâu?	Chi phí thật của từng loại lỗi (quyết định kinh doanh).
ROC vẽ gì?	TPR (recall) theo FPR ở mọi ngưỡng.
AUC = 0,8 nghĩa là?	80% số cặp (dương, âm) được mô hình xếp hạng đúng.
Chọn ngưỡng trên tập nào?	Validation (hoặc cross-validation), không phải test.

Bài tiếp theo: 2.6 – Bias–Variance, Overfitting/Underfitting, Regularization L1/L2.